

EXP – 3

COMPARISON OF SGD WITH MOMENTUM VS ADAM OPTIMIZER

AIM

To implement and compare **Stochastic Gradient Descent (SGD) with Momentum** and **Adam optimizer** using a Multi-Layer Perceptron (MLP) on the **CIFAR-10 dataset**, and analyze their convergence using loss and accuracy.

OBJECTIVES

1. To understand optimization algorithms used in deep learning.
2. To implement SGD with Momentum and Adam optimizers.
3. To train an MLP using both optimizers.
4. To compare their loss and accuracy.
5. To visualize their convergence patterns.

DATASET USED

CIFAR-10 Dataset

- 60,000 color images
- Image size: 32×32
- 10 different classes
- 50,000 training images
- 10,000 testing images

TOOLS AND LIBRARIES USED

1. Python 3.x
2. PyTorch
3. Torchvision
4. Matplotlib
5. Google Colab

ALGORITHM

1. Import the required PyTorch, Torchvision and Matplotlib libraries.
2. Select GPU if available.
3. Load the CIFAR-10 training dataset.
4. Normalize the images.
5. Define an MLP model with input, hidden and output layers.
6. Train one model using SGD with Momentum.
7. Train another model using Adam optimizer.
8. Calculate loss and accuracy for each epoch.
9. Store the training loss and accuracy.
10. Plot and compare the loss and accuracy of both optimizers.
11. Analyze the convergence behavior.

PROCEDURE

1. Open **Google Colab**.
2. Create a new notebook.
3. Import the required libraries.
4. Load and preprocess the CIFAR-10 dataset.
5. Define the MLP model.
6. Create the SGD with Momentum optimizer.
7. Train the model and record loss and accuracy.
8. Create another MLP model using Adam optimizer.
9. Train the model and record loss and accuracy.
10. Plot the loss and accuracy comparison.
11. Observe the convergence of both optimizers.

PROGRAM

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True
)

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.net = nn.Sequential(
            nn.Linear(3 * 32 * 32, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )

    def forward(self, x):
        return self.net(self.flatten(x))
```

```

def train(model, optimizer, epochs=10):
    model.to(device)
    loss_fn = nn.CrossEntropyLoss()
    losses = []
    accuracies = []

    for epoch in range(epochs):
        total_loss = 0
        correct = 0
        total = 0

        model.train()

        for images, labels in trainloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = loss_fn(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            predictions = outputs.argmax(1)
            total += labels.size(0)
            correct += (predictions == labels).sum().item()

        avg_loss = total_loss / len(trainloader)
        accuracy = 100 * correct / total

        losses.append(avg_loss)
        accuracies.append(accuracy)

        print(f'Epoch {epoch + 1}/10 | Loss: {avg_loss:.4f} | Accuracy: {accuracy:.2f}%')

    return losses, accuracies

model_sgd = MLP()
sgd = optim.SGD(model_sgd.parameters(), lr=0.01, momentum=0.9)
losses_sgd, acc_sgd = train(model_sgd, sgd)

model_adam = MLP()
adam = optim.Adam(model_adam.parameters(), lr=0.001)
losses_adam, acc_adam = train(model_adam, adam)

epochs = range(1, 11)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(epochs, losses_sgd, marker="o", label="SGD + Momentum")
plt.plot(epochs, losses_adam, marker="o", label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Comparison on CIFAR-10")
plt.legend()

```

```

plt.subplot(1, 2, 2)
plt.plot(epochs, acc_sgd, marker="o", label="SGD + Momentum")
plt.plot(epochs, acc_adam, marker="o", label="Adam")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison on CIFAR-10")
plt.legend()

plt.tight_layout()
plt.show()

print("\nFinal Comparison")
print(f"SGD + Momentum - Loss: {losses_sgd[-1]:.4f}, Accuracy: {acc_sgd[-1]:.2f}%")
print(f"Adam - Loss: {losses_adam[-1]:.4f}, Accuracy: {acc_adam[-1]:.2f}%")

```

OBSERVATION

During training, the loss and accuracy of both models are recorded for each epoch.

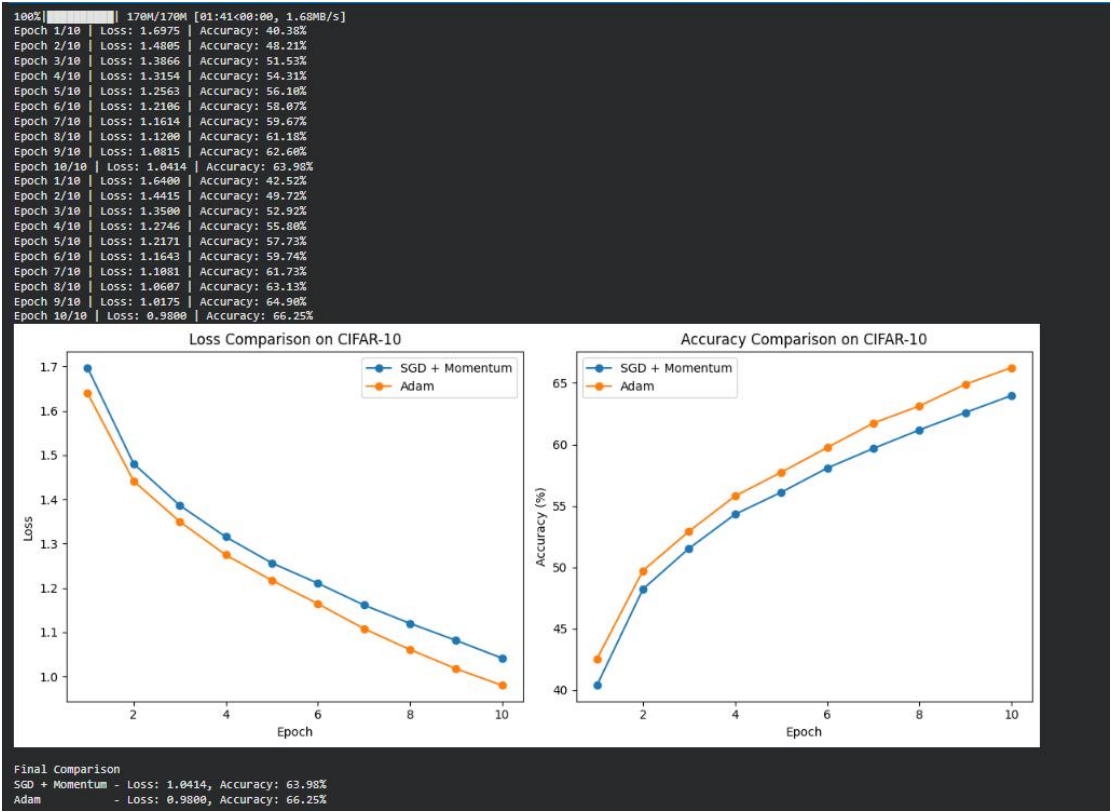
Epoch	SGD + Momentum Loss	SGD + Momentum Accuracy	Adam Loss	Adam Accuracy
1	Observed	Observed	Observed	Observed
2	Observed	Observed	Observed	Observed
3	Observed	Observed	Observed	Observed
4	Observed	Observed	Observed	Observed
5	Observed	Observed	Observed	Observed
6	Observed	Observed	Observed	Observed
7	Observed	Observed	Observed	Observed
8	Observed	Observed	Observed	Observed
9	Observed	Observed	Observed	Observed
10	Observed	Observed	Observed	Observed

GRAPH OBSERVATION

Two graphs are obtained:

1. **Loss Comparison** – compares the decrease in loss for SGD with Momentum and Adam.
2. **Accuracy Comparison** – compares the increase in accuracy for both optimizers.

OUTPUT:



RESULT

The **SGD with Momentum** and **Adam** optimizers were successfully implemented and trained using an MLP on the CIFAR-10 dataset. The loss and accuracy curves were plotted to compare their convergence behavior.

Adam generally converges faster in the initial training stages, while SGD with Momentum provides gradual and stable learning.

